

UNIVERSIDADE FEDERAL DE CIÊNCIA E INOVAÇÃO DE MINAS GERAIS

CAROLINE ELIZABETH LEROY ALVES

CLARA LIBERAL BARBI

JULIA MEIRELES GONTIJO

ANALISADOR LÉXICO E TABELA DE SÍMBOLOS

Relatório da prática de Compiladores - etapa 1

Belo Horizonte

2026

O compilador foi desenvolvido em linguagem Java e executamos por meio do terminal. Para compilar os arquivos fontes do projeto utilizamos o comando:

```
javac -source 1.8 -target 1.8 *.java
```

Para executar o compilador passando um arquivo de código-fonte como entrada, utilizamos o comando:

```
java Main <caminho_do_arquivo_fonte>
```

Exemplo: java Main Testes/Teste1.txt

A implementação do analisador léxico e da tabela de símbolos foi dividido entre as seguintes classes:

- **Main:** Controladora da execução. Responsável por receber o arquivo de entrada por parâmetro, instanciar o analisador léxico, iterar sobre o fluxo de tokens até o fim do arquivo exibindo linha a linha os resultados e acionar a impressão da Tabela de Símbolos ao término.
- **Lexer:** Núcleo do analisador léxico. Realiza a leitura caractere por caractere do código-fonte, gerencia a contagem de linhas para rastreamento de erros, descarta espaços em branco e ignora comentários (tanto de linha com % quanto multilinha com {...}). Reconhece operadores simples, operadores compostos, literais entre aspas, constantes de caracteres, números inteiros/reaais, identificadores e palavras reservadas.
- **Token:** Classe que representa os símbolos terminais reconhecidos, contendo o atributo tag correspondente.
- **Word:** Lida com identificadores e palavras reservadas. Mantém o lexema correspondente e gerencia a estrutura da Tabela de Símbolos implementada por meio de uma Hashtable.
- **Num, Real, CharConst e Literal:** Classes auxiliares que estendem tokens para armazenar os respectivos valores de constantes numéricas, caracteres e strings.

TESTE 1

Programa fonte analisado:

```
program teste1
{*Calculo de perimetro e area*}
base, altura: float;
perimetro: float;
area, la is float;
begin
write ("Entre com a altura do retangulo:");
read (altura);
write("Entre com a base do retangulo:");
read (base);
area = base * altura;
perimetro = 2 * base + 2 * altura;
write ("Area: ");
write (area);
```

```
write ("Perímetro: ");
write (area);
end.
```

Resultado: SUCESSO!

----- SEQUENCIA DE TOKENS -----

Linha 1 | Token Tag: 279 | Lexema/Valor: [programa]
Linha 1 | Token Tag: 279 | Lexema/Valor: [teste1]
Linha 4 | Token Tag: 279 | Lexema/Valor: [base]
Linha 4 | Token Tag: 44 | Lexema/Valor: [.,]
Linha 4 | Token Tag: 279 | Lexema/Valor: [altura]
Linha 4 | Token Tag: 58 | Lexema/Valor: [;]
Linha 4 | Token Tag: 262 | Lexema/Valor: [float]
Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
Linha 5 | Token Tag: 279 | Lexema/Valor: [perimetro]
Linha 5 | Token Tag: 58 | Lexema/Valor: [;]
Linha 5 | Token Tag: 262 | Lexema/Valor: [float]
Linha 5 | Token Tag: 59 | Lexema/Valor: [;]
Linha 6 | Token Tag: 279 | Lexema/Valor: [area]
Linha 6 | Token Tag: 44 | Lexema/Valor: [.,]
Linha 6 | Token Tag: 278 | Lexema/Valor: [1]
Linha 6 | Token Tag: 279 | Lexema/Valor: [a]
Linha 6 | Token Tag: 279 | Lexema/Valor: [is]
Linha 6 | Token Tag: 262 | Lexema/Valor: [float]
Linha 6 | Token Tag: 59 | Lexema/Valor: [;]
Linha 8 | Token Tag: 257 | Lexema/Valor: [begin]
Linha 9 | Token Tag: 279 | Lexema/Valor: [write]
Linha 9 | Token Tag: 40 | Lexema/Valor: [(]
Linha 9 | Token Tag: 282 | Lexema/Valor: ["Entre com a altura do retangulo:"]
Linha 9 | Token Tag: 41 | Lexema/Valor: [)]
Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
Linha 10 | Token Tag: 279 | Lexema/Valor: [read]
Linha 10 | Token Tag: 40 | Lexema/Valor: [(]
Linha 10 | Token Tag: 279 | Lexema/Valor: [altura]
Linha 10 | Token Tag: 41 | Lexema/Valor: [)]
Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
Linha 11 | Token Tag: 279 | Lexema/Valor: [write]
Linha 11 | Token Tag: 40 | Lexema/Valor: [(]
Linha 11 | Token Tag: 282 | Lexema/Valor: ["Entre com a base do retangulo:"]
Linha 11 | Token Tag: 41 | Lexema/Valor: [)]
Linha 11 | Token Tag: 59 | Lexema/Valor: [;]
Linha 12 | Token Tag: 279 | Lexema/Valor: [read]
Linha 12 | Token Tag: 40 | Lexema/Valor: [(]
Linha 12 | Token Tag: 279 | Lexema/Valor: [base]

Linha 12 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 12 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 14 | Token Tag: 279 | Lexema/Valor: [area]
 Linha 14 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 14 | Token Tag: 279 | Lexema/Valor: [base]
 Linha 14 | Token Tag: 42 | Lexema/Valor: [*]
 Linha 14 | Token Tag: 279 | Lexema/Valor: [altura]
 Linha 14 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 15 | Token Tag: 279 | Lexema/Valor: [perimetro]
 Linha 15 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 15 | Token Tag: 278 | Lexema/Valor: [2]
 Linha 15 | Token Tag: 42 | Lexema/Valor: [*]
 Linha 15 | Token Tag: 279 | Lexema/Valor: [base]
 Linha 15 | Token Tag: 43 | Lexema/Valor: [+]
 Linha 15 | Token Tag: 278 | Lexema/Valor: [2]
 Linha 15 | Token Tag: 42 | Lexema/Valor: [*]
 Linha 15 | Token Tag: 279 | Lexema/Valor: [altura]
 Linha 15 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 17 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 17 | Token Tag: 282 | Lexema/Valor: ["Area: "
 Linha 17 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 18 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 18 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 18 | Token Tag: 279 | Lexema/Valor: [area]
 Linha 18 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 18 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 19 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 19 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 19 | Token Tag: 282 | Lexema/Valor: ["PerÃmetro: "
 Linha 19 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 19 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 20 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 20 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 20 | Token Tag: 279 | Lexema/Valor: [area]
 Linha 20 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 20 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 21 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 21 | Token Tag: 46 | Lexema/Valor: [.]

TABELA DE SIMBOLOS

Lexema: int | Tag: 260
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: altura | Tag: 279
 Lexema: teste1 | Tag: 279
 Lexema: base | Tag: 279

Lexema: is | Tag: 279
Lexema: program | Tag: 256
Lexema: in | Tag: 270
Lexema: repeat | Tag: 268
Lexema: programa | Tag: 279
Lexema: if | Tag: 263
Lexema: while | Tag: 266
Lexema: do | Tag: 267
Lexema: perimetro | Tag: 279
Lexema: read | Tag: 279
Lexema: out | Tag: 271
Lexema: else | Tag: 265
Lexema: write | Tag: 279
Lexema: area | Tag: 279
Lexema: then | Tag: 264
Lexema: char | Tag: 261
Lexema: begin | Tag: 257
Lexema: a | Tag: 279
Lexema: float | Tag: 262

TESTE 2

Programa fonte analisado:

```
program teste2
a, b, 1c: int;
d, _var: float;
teste2 = 1;
read (a);
b = a * a;
c = b + a/2 * (35/b);
write (c);
val := 34.;
c = val + 2.2 + a;
write (val)
end.
```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 2): identificador mal formado (nao pode iniciar com numero).

Correção 1:

```

program teste2
  a, b, valc :int;
  d, _var: float;
  teste2 = 1;
  Read (a);
  b = a * a;
  c = b + a/2 * (35/b);
  write (c);
  val := 34.;
  c = val + 2.2 + a;
  write (val)
end.

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 9): constante real mal formada.

Correção 2:

```

program teste2
  a, b, valc :int;
  d, _var: float;
  teste2 = 1;
  Read (a);
  b = a * a;
  c = b + a/2 * (35/b);
  write (c);
  val := 34.0;
  c = val + 2.2 + a;
  write (val)
end.

```

Resultado: SUCESSO!

----- SEQUENCIA DE TOKENS -----

Linha 1 | Token Tag: 256 | Lexema/Valor: [program]
 Linha 1 | Token Tag: 279 | Lexema/Valor: [teste2]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [a]

Linha 2 | Token Tag: 44 | Lexema/Valor: [,]
Linha 2 | Token Tag: 279 | Lexema/Valor: [b]
Linha 2 | Token Tag: 44 | Lexema/Valor: [,]
Linha 2 | Token Tag: 279 | Lexema/Valor: [valc]
Linha 2 | Token Tag: 58 | Lexema/Valor: [:]
Linha 2 | Token Tag: 260 | Lexema/Valor: [int]
Linha 2 | Token Tag: 59 | Lexema/Valor: [;]
Linha 3 | Token Tag: 279 | Lexema/Valor: [d]
Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
Linha 3 | Token Tag: 279 | Lexema/Valor: [_var]
Linha 3 | Token Tag: 58 | Lexema/Valor: [:]
Linha 3 | Token Tag: 262 | Lexema/Valor: [float]
Linha 3 | Token Tag: 59 | Lexema/Valor: [;]
Linha 4 | Token Tag: 279 | Lexema/Valor: [teste2]
Linha 4 | Token Tag: 61 | Lexema/Valor: [=]
Linha 4 | Token Tag: 278 | Lexema/Valor: [1]
Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
Linha 5 | Token Tag: 279 | Lexema/Valor: [Read]
Linha 5 | Token Tag: 40 | Lexema/Valor: [(
Linha 5 | Token Tag: 279 | Lexema/Valor: [a]
Linha 5 | Token Tag: 41 | Lexema/Valor: [)]
Linha 5 | Token Tag: 59 | Lexema/Valor: [;]
Linha 6 | Token Tag: 279 | Lexema/Valor: [b]
Linha 6 | Token Tag: 61 | Lexema/Valor: [=]
Linha 6 | Token Tag: 279 | Lexema/Valor: [a]
Linha 6 | Token Tag: 42 | Lexema/Valor: [*]
Linha 6 | Token Tag: 279 | Lexema/Valor: [a]
Linha 6 | Token Tag: 59 | Lexema/Valor: [;]
Linha 7 | Token Tag: 279 | Lexema/Valor: [c]
Linha 7 | Token Tag: 61 | Lexema/Valor: [=]
Linha 7 | Token Tag: 279 | Lexema/Valor: [b]
Linha 7 | Token Tag: 43 | Lexema/Valor: [+]
Linha 7 | Token Tag: 279 | Lexema/Valor: [a]
Linha 7 | Token Tag: 47 | Lexema/Valor: [/]
Linha 7 | Token Tag: 278 | Lexema/Valor: [2]
Linha 7 | Token Tag: 42 | Lexema/Valor: [*]
Linha 7 | Token Tag: 40 | Lexema/Valor: [(
Linha 7 | Token Tag: 278 | Lexema/Valor: [35]
Linha 7 | Token Tag: 47 | Lexema/Valor: [/]
Linha 7 | Token Tag: 279 | Lexema/Valor: [b]
Linha 7 | Token Tag: 41 | Lexema/Valor: [)]
Linha 7 | Token Tag: 59 | Lexema/Valor: [;]
Linha 8 | Token Tag: 279 | Lexema/Valor: [write]
Linha 8 | Token Tag: 40 | Lexema/Valor: [(
Linha 8 | Token Tag: 279 | Lexema/Valor: [c]
Linha 8 | Token Tag: 41 | Lexema/Valor: [)]
Linha 8 | Token Tag: 59 | Lexema/Valor: [;]
Linha 9 | Token Tag: 279 | Lexema/Valor: [val]

Linha 9 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 9 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 9 | Token Tag: 280 | Lexema/Valor: [34.0]
 Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 10 | Token Tag: 279 | Lexema/Valor: [c]
 Linha 10 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 10 | Token Tag: 279 | Lexema/Valor: [val]
 Linha 10 | Token Tag: 43 | Lexema/Valor: [+]
 Linha 10 | Token Tag: 280 | Lexema/Valor: [2.2]
 Linha 10 | Token Tag: 43 | Lexema/Valor: [+]
 Linha 10 | Token Tag: 279 | Lexema/Valor: [a]
 Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 11 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 11 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 11 | Token Tag: 279 | Lexema/Valor: [val]
 Linha 11 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 12 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 12 | Token Tag: 46 | Lexema/Valor: [.]

TABELA DE SIMBOLOS

Lexema: _var | Tag: 279
 Lexema: int | Tag: 260
 Lexema: teste2 | Tag: 279
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: program | Tag: 256
 Lexema: in | Tag: 270
 Lexema: repeat | Tag: 268
 Lexema: if | Tag: 263
 Lexema: while | Tag: 266
 Lexema: do | Tag: 267
 Lexema: out | Tag: 271
 Lexema: else | Tag: 265
 Lexema: write | Tag: 279
 Lexema: then | Tag: 264
 Lexema: d | Tag: 279
 Lexema: char | Tag: 261
 Lexema: c | Tag: 279
 Lexema: valc | Tag: 279
 Lexema: begin | Tag: 257
 Lexema: b | Tag: 279
 Lexema: a | Tag: 279
 Lexema: Read | Tag: 279
 Lexema: float | Tag: 262
 Lexema: val | Tag: 279

TESTE 3

Programa fonte analisado:

```
program
  a, aux : int;
  b : float

  begin
    b = 0;
    read (a);
    read(b);
    if (a>b) then
      aux = b;
      b = a;
      a = aux
    end;
    write(a);
    write(b);

    if (a%2 == 0) write ("Par");
    else write ("Impar");
```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 16): literal nao fechado.

Correção:

```

program teste3
a, aux : int;
b : float;
begin
b = 0;
read (a);
read(b);
if (a>b) then
aux = b;
b = a;
a = aux;
end;
write(a);
write(b);
if (a%2 == 0) then
write ("Par");
else
write ("Impar");
end
end.

```

Resultado: SUCESSO!

----- SEQUENCIA DE TOKENS -----

Linha 1 | Token Tag: 256 | Lexema/Valor: [program]
 Linha 1 | Token Tag: 279 | Lexema/Valor: [teste3]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [a]
 Linha 2 | Token Tag: 44 | Lexema/Valor: [,]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [aux]
 Linha 2 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 2 | Token Tag: 260 | Lexema/Valor: [int]
 Linha 2 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 3 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 3 | Token Tag: 262 | Lexema/Valor: [float]
 Linha 3 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 4 | Token Tag: 257 | Lexema/Valor: [begin]
 Linha 5 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 5 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 5 | Token Tag: 278 | Lexema/Valor: [0]
 Linha 5 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 6 | Token Tag: 279 | Lexema/Valor: [read]
 Linha 6 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 6 | Token Tag: 279 | Lexema/Valor: [a]

Linha 6 | Token Tag: 41 | Lexema/Valor: [)]
Linha 6 | Token Tag: 59 | Lexema/Valor: [;]
Linha 7 | Token Tag: 279 | Lexema/Valor: [read]
Linha 7 | Token Tag: 40 | Lexema/Valor: [(]
Linha 7 | Token Tag: 279 | Lexema/Valor: [b]
Linha 7 | Token Tag: 41 | Lexema/Valor: [)]
Linha 7 | Token Tag: 59 | Lexema/Valor: [;]
Linha 8 | Token Tag: 263 | Lexema/Valor: [if]
Linha 8 | Token Tag: 40 | Lexema/Valor: [(]
Linha 8 | Token Tag: 279 | Lexema/Valor: [a]
Linha 8 | Token Tag: 62 | Lexema/Valor: [>]
Linha 8 | Token Tag: 279 | Lexema/Valor: [b]
Linha 8 | Token Tag: 41 | Lexema/Valor: [)]
Linha 8 | Token Tag: 264 | Lexema/Valor: [then]
Linha 9 | Token Tag: 279 | Lexema/Valor: [aux]
Linha 9 | Token Tag: 61 | Lexema/Valor: [=]
Linha 9 | Token Tag: 279 | Lexema/Valor: [b]
Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
Linha 10 | Token Tag: 279 | Lexema/Valor: [b]
Linha 10 | Token Tag: 61 | Lexema/Valor: [=]
Linha 10 | Token Tag: 279 | Lexema/Valor: [a]
Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
Linha 11 | Token Tag: 279 | Lexema/Valor: [a]
Linha 11 | Token Tag: 61 | Lexema/Valor: [=]
Linha 11 | Token Tag: 279 | Lexema/Valor: [aux]
Linha 11 | Token Tag: 59 | Lexema/Valor: [;]
Linha 12 | Token Tag: 258 | Lexema/Valor: [end]
Linha 12 | Token Tag: 59 | Lexema/Valor: [;]
Linha 13 | Token Tag: 279 | Lexema/Valor: [write]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 279 | Lexema/Valor: [a]
Linha 13 | Token Tag: 41 | Lexema/Valor: [)]
Linha 13 | Token Tag: 59 | Lexema/Valor: [;]
Linha 14 | Token Tag: 279 | Lexema/Valor: [write]
Linha 14 | Token Tag: 40 | Lexema/Valor: [(]
Linha 14 | Token Tag: 279 | Lexema/Valor: [b]
Linha 14 | Token Tag: 41 | Lexema/Valor: [)]
Linha 14 | Token Tag: 59 | Lexema/Valor: [;]
Linha 15 | Token Tag: 263 | Lexema/Valor: [if]
Linha 15 | Token Tag: 40 | Lexema/Valor: [(]
Linha 15 | Token Tag: 279 | Lexema/Valor: [a]
Linha 15 | Token Tag: 37 | Lexema/Valor: [%]
Linha 15 | Token Tag: 278 | Lexema/Valor: [2]
Linha 15 | Token Tag: 288 | Lexema/Valor: [==]
Linha 15 | Token Tag: 278 | Lexema/Valor: [0]
Linha 15 | Token Tag: 41 | Lexema/Valor: [)]
Linha 15 | Token Tag: 264 | Lexema/Valor: [then]
Linha 16 | Token Tag: 279 | Lexema/Valor: [write]

Linha 16 | Token Tag: 40 | Lexema/Valor: [(
 Linha 16 | Token Tag: 282 | Lexema/Valor: ["Par"]
 Linha 16 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 16 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 17 | Token Tag: 265 | Lexema/Valor: [else]
 Linha 18 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 18 | Token Tag: 40 | Lexema/Valor: [(
 Linha 18 | Token Tag: 282 | Lexema/Valor: ["Impar"]
 Linha 18 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 18 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 19 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 20 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 20 | Token Tag: 46 | Lexema/Valor: [.]

TABELA DE SIMBOLOS

Lexema: int | Tag: 260
 Lexema: teste3 | Tag: 279
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: program | Tag: 256
 Lexema: in | Tag: 270
 Lexema: repeat | Tag: 268
 Lexema: if | Tag: 263
 Lexema: while | Tag: 266
 Lexema: do | Tag: 267
 Lexema: read | Tag: 279
 Lexema: out | Tag: 271
 Lexema: else | Tag: 265
 Lexema: write | Tag: 279
 Lexema: then | Tag: 264
 Lexema: char | Tag: 261
 Lexema: begin | Tag: 257
 Lexema: b | Tag: 279
 Lexema: aux | Tag: 279
 Lexema: a | Tag: 279
 Lexema: float | Tag: 262

TESTE 4

Programa fonte analisado:

```

programa teste4

(* Teste4 do meu compilador

    pontuacao, pontuacaoMaxima, disponibilidade : inteiro;
    pontuacaoMinima : char;

begin
    pontuacaoMinima = 50;
    pontuacaoMaxima = 100;
    write("Pontuacao do candidato: ");
    read(pontuacao);
    write("Disponibilidade do candidato: ");
    read(disponibilidade);

    while (pontuacao>0 && (pontuacao<=pontuacaoMaxima) do
        if ((pontuação > pontuacaoMinima) && (disponibilidade==1)) then
            write("Candidato aprovado. ")
        else
            write("Candidato reprovado. ")
        end

        write("Pontuacao do candidato: ");
        read(pontuacao);
        write("Disponibilidade do
            candidato: ");
        read(disponibilidade);

    end
end

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 2): comentario nao fechado.

Correção 1:

```

programa teste4
{
  (* Teste4 do meu compilador *)
  pontuacao, pontuacaoMaxima, disponibilidade : inteiro;
  pontuacaoMinima : char;
begin
  pontuacaoMinima = 50;
  pontuacaoMaxima = 100;
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do candidato: ");
  read(disponibilidade);

  while (pontuacao>0 && (pontuacao<=pontuacaoMaxima) do
  if ((pontuação > pontuacaoMinima) && (disponibilidade==1)) then
  write("Candidato aprovado. ")
  else
  write("Candidato reprovado. ")
  end
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do
  candidato: ");
  read(disponibilidade);
  end
end
end

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 14): caractere invalido 'Ã'.

Correção 2:

```

programa teste4
{
  (* Teste4 do meu compilador *)
  pontuacao, pontuacaoMaxima, disponibilidade : inteiro;
  pontuacaoMinima : char;
begin
  pontuacaoMinima = 50;
  pontuacaoMaxima = 100;
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do candidato: ");
  read(disponibilidade);

  while (pontuacao>0 && (pontuacao<=pontuacaoMaxima) do
  if ((pontuacao > pontuacaoMinima) && (disponibilidade==1)) then
  write("Candidato aprovado. ")
  else
  write("Candidato reprovado. ")
  end
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do
  candidato: ");
  read(disponibilidade);
  end
end
end

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 21): literal nao fechado.

Correção 3:

```
programa teste4
{
  /* Teste4 do meu compilador */
  pontuacao, pontuacaoMaxima, disponibilidade : inteiro;
  pontuacaoMinima : char;
begin
  pontuacaoMinima = 50;
  pontuacaoMaxima = 100;
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do candidato: ");
  read(disponibilidade);

  while (pontuacao>0 && (pontuacao<=pontuacaoMaxima) do
  if ((pontuacao > pontuacaoMinima) && (disponibilidade==1)) then
  write("Candidato aprovado. ")
  else
  write("Candidato reprovado. ")
  end
  write("Pontuacao do candidato: ");
  read(pontuacao);
  write("Disponibilidade do candidato: ");
  read(disponibilidade);
  end
end
```

Resultado: SUCESSO!

----- SEQUENCIA DE TOKENS -----

Linha 1 | Token Tag: 279 | Lexema/Valor: [programa]
Linha 1 | Token Tag: 279 | Lexema/Valor: [teste4]
Linha 3 | Token Tag: 279 | Lexema/Valor: [pontuacao]
Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
Linha 3 | Token Tag: 279 | Lexema/Valor: [pontuacaoMaxima]
Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
Linha 3 | Token Tag: 279 | Lexema/Valor: [disponibilidade]
Linha 3 | Token Tag: 58 | Lexema/Valor: [:]
Linha 3 | Token Tag: 279 | Lexema/Valor: [inteiro]
Linha 3 | Token Tag: 59 | Lexema/Valor: [;]
Linha 4 | Token Tag: 279 | Lexema/Valor: [pontuacaoMinima]
Linha 4 | Token Tag: 58 | Lexema/Valor: [:]
Linha 4 | Token Tag: 261 | Lexema/Valor: [char]
Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
Linha 5 | Token Tag: 257 | Lexema/Valor: [begin]
Linha 6 | Token Tag: 279 | Lexema/Valor: [pontuacaoMinima]
Linha 6 | Token Tag: 61 | Lexema/Valor: [=]
Linha 6 | Token Tag: 278 | Lexema/Valor: [50]
Linha 6 | Token Tag: 59 | Lexema/Valor: [;]

Linha 7 | Token Tag: 279 | Lexema/Valor: [pontuacaoMaxima]
Linha 7 | Token Tag: 61 | Lexema/Valor: [=]
Linha 7 | Token Tag: 278 | Lexema/Valor: [100]
Linha 7 | Token Tag: 59 | Lexema/Valor: [;]
Linha 8 | Token Tag: 279 | Lexema/Valor: [write]
Linha 8 | Token Tag: 40 | Lexema/Valor: [(]
Linha 8 | Token Tag: 282 | Lexema/Valor: ["Pontuacao do candidato: "
Linha 8 | Token Tag: 41 | Lexema/Valor: [])
Linha 8 | Token Tag: 59 | Lexema/Valor: [;]
Linha 9 | Token Tag: 279 | Lexema/Valor: [read]
Linha 9 | Token Tag: 40 | Lexema/Valor: [(]
Linha 9 | Token Tag: 279 | Lexema/Valor: [pontuacao]
Linha 9 | Token Tag: 41 | Lexema/Valor: [])
Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
Linha 10 | Token Tag: 279 | Lexema/Valor: [write]
Linha 10 | Token Tag: 40 | Lexema/Valor: [(]
Linha 10 | Token Tag: 282 | Lexema/Valor: ["Disponibilidade do candidato: "
Linha 10 | Token Tag: 41 | Lexema/Valor: [])
Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
Linha 11 | Token Tag: 279 | Lexema/Valor: [read]
Linha 11 | Token Tag: 40 | Lexema/Valor: [(]
Linha 11 | Token Tag: 279 | Lexema/Valor: [disponibilidade]
Linha 11 | Token Tag: 41 | Lexema/Valor: [])
Linha 11 | Token Tag: 59 | Lexema/Valor: [;]
Linha 13 | Token Tag: 266 | Lexema/Valor: [while]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 279 | Lexema/Valor: [pontuacao]
Linha 13 | Token Tag: 62 | Lexema/Valor: [>]
Linha 13 | Token Tag: 278 | Lexema/Valor: [0]
Linha 13 | Token Tag: 292 | Lexema/Valor: [&&]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 279 | Lexema/Valor: [pontuacao]
Linha 13 | Token Tag: 290 | Lexema/Valor: [<=]
Linha 13 | Token Tag: 279 | Lexema/Valor: [pontuacaoMaxima]
Linha 13 | Token Tag: 41 | Lexema/Valor: [])
Linha 13 | Token Tag: 267 | Lexema/Valor: [do]
Linha 14 | Token Tag: 263 | Lexema/Valor: [if]
Linha 14 | Token Tag: 40 | Lexema/Valor: [(]
Linha 14 | Token Tag: 40 | Lexema/Valor: [(]
Linha 14 | Token Tag: 279 | Lexema/Valor: [pontuacao]
Linha 14 | Token Tag: 62 | Lexema/Valor: [>]
Linha 14 | Token Tag: 279 | Lexema/Valor: [pontuacaoMinima]
Linha 14 | Token Tag: 41 | Lexema/Valor: [])
Linha 14 | Token Tag: 292 | Lexema/Valor: [&&]
Linha 14 | Token Tag: 40 | Lexema/Valor: [(]
Linha 14 | Token Tag: 279 | Lexema/Valor: [disponibilidade]
Linha 14 | Token Tag: 288 | Lexema/Valor: [==]
Linha 14 | Token Tag: 278 | Lexema/Valor: [1]

Linha 14 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 14 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 14 | Token Tag: 264 | Lexema/Valor: [then]
 Linha 15 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 15 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 15 | Token Tag: 282 | Lexema/Valor: ["Candidato aprovado. "]
 Linha 15 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 16 | Token Tag: 265 | Lexema/Valor: [else]
 Linha 17 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 17 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 17 | Token Tag: 282 | Lexema/Valor: ["Candidato reprovado. "]
 Linha 17 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 18 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 19 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 19 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 19 | Token Tag: 282 | Lexema/Valor: ["Pontuacao do candidato: "]
 Linha 19 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 19 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 20 | Token Tag: 279 | Lexema/Valor: [read]
 Linha 20 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 20 | Token Tag: 279 | Lexema/Valor: [pontuacao]
 Linha 20 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 20 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 21 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 21 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 21 | Token Tag: 282 | Lexema/Valor: ["Disponibilidade do candidato: "]
 Linha 21 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 21 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 22 | Token Tag: 279 | Lexema/Valor: [read]
 Linha 22 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 22 | Token Tag: 279 | Lexema/Valor: [disponibilidade]
 Linha 22 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 22 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 23 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 24 | Token Tag: 258 | Lexema/Valor: [end]

TABELA DE SIMBOLOS

Lexema: int | Tag: 260
 Lexema: disponibilidade | Tag: 279
 Lexema: teste4 | Tag: 279
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: inteiro | Tag: 279
 Lexema: program | Tag: 256
 Lexema: in | Tag: 270
 Lexema: repeat | Tag: 268

Lexema: programa | Tag: 279
Lexema: pontuacao | Tag: 279
Lexema: if | Tag: 263
Lexema: while | Tag: 266
Lexema: pontuacaoMinima | Tag: 279
Lexema: do | Tag: 267
Lexema: read | Tag: 279
Lexema: out | Tag: 271
Lexema: pontuacaoMaxima | Tag: 279
Lexema: else | Tag: 265
Lexema: write | Tag: 279
Lexema: then | Tag: 264
Lexema: char | Tag: 261
Lexema: begin | Tag: 257
Lexema: float | Tag: 262

TESTE 5

Programa fonte analisado:

```

{* Teste do meu compilador *}

program teste5
  a, b, c, maior : int;
  opc : int;

begin
  repeat
    write("A: ");
    read(a);
    write("B: ");
    read(b);
    write("C: ");
    read(c);

    if ( (a>b) && (a>c) ) end
      maior = a

    else
      if (b>c) then
        maior = b;

        else
          maior = c
        end
      end;
    write("Maior valor: ");
    write (maior);
    write (" Continuar?(S/N) ");
    read(outro);
  until (opc == 'N' || opc == 'n')
end.

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 24): literal nao fechado.

Correção 1:

```

{* Teste do meu compilador *}
program teste5
  a, b, c, maior : int;
  opc : int;
begin
  repeat
    write("A: ");
    read(a);
    write("B: ");
    read(b);
    write("C: ");
    read(c);
    if ( (a>b) && (a>c) ) end
    maior = a

  else
    if (b>c) then
      maior = b;

    else
      maior = c
    end
  end;
  write("Maior valor: ");
  write (maior);
  write (" Continuar?(S/N) ");
  read(outro);
  until (opc == 'N' || opc == 'n')
end.

```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 28): constante de caractere deve possuir apenas um caractere.

Correção 2:

```

{* Teste do meu compilador *}
program teste5
  a, b, c, maior : int;
  opc : int;
begin
  repeat
    write("A: ");
    read(a);
    write("B: ");
    read(b);
    write("C: ");
    read(c);
    if ( (a>b) && (a>c) ) end
    maior = a

  else
    if (b>c) then
      maior = b;

    else
      maior = c
    end
  end;
  write("Maior valor: ");
  write (maior);
  write (" Continuar?(S/N) ");
  read(outro);
  until (opc == 'N' || opc == 'n')
end.

```

Resultado: SUCESSO!

SEQUENCIA DE TOKENS

Linha 2 | Token Tag: 256 | Lexema/Valor: [program]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [teste5]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [a]
 Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [c]
 Linha 3 | Token Tag: 44 | Lexema/Valor: [,]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [maior]
 Linha 3 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 3 | Token Tag: 260 | Lexema/Valor: [int]
 Linha 3 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 4 | Token Tag: 279 | Lexema/Valor: [opc]

Linha 4 | Token Tag: 58 | Lexema/Valor: [:]
Linha 4 | Token Tag: 260 | Lexema/Valor: [int]
Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
Linha 5 | Token Tag: 257 | Lexema/Valor: [begin]
Linha 6 | Token Tag: 268 | Lexema/Valor: [repeat]
Linha 7 | Token Tag: 279 | Lexema/Valor: [write]
Linha 7 | Token Tag: 40 | Lexema/Valor: [(]
Linha 7 | Token Tag: 282 | Lexema/Valor: ["A: "
Linha 7 | Token Tag: 41 | Lexema/Valor: [)]
Linha 7 | Token Tag: 59 | Lexema/Valor: [;]
Linha 8 | Token Tag: 279 | Lexema/Valor: [read]
Linha 8 | Token Tag: 40 | Lexema/Valor: [(]
Linha 8 | Token Tag: 279 | Lexema/Valor: [a]
Linha 8 | Token Tag: 41 | Lexema/Valor: [)]
Linha 8 | Token Tag: 59 | Lexema/Valor: [;]
Linha 9 | Token Tag: 279 | Lexema/Valor: [write]
Linha 9 | Token Tag: 40 | Lexema/Valor: [(]
Linha 9 | Token Tag: 282 | Lexema/Valor: ["B: "
Linha 9 | Token Tag: 41 | Lexema/Valor: [)]
Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
Linha 10 | Token Tag: 279 | Lexema/Valor: [read]
Linha 10 | Token Tag: 40 | Lexema/Valor: [(]
Linha 10 | Token Tag: 279 | Lexema/Valor: [b]
Linha 10 | Token Tag: 41 | Lexema/Valor: [)]
Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
Linha 11 | Token Tag: 279 | Lexema/Valor: [write]
Linha 11 | Token Tag: 40 | Lexema/Valor: [(]
Linha 11 | Token Tag: 282 | Lexema/Valor: ["C: "
Linha 11 | Token Tag: 41 | Lexema/Valor: [)]
Linha 11 | Token Tag: 59 | Lexema/Valor: [;]
Linha 12 | Token Tag: 279 | Lexema/Valor: [read]
Linha 12 | Token Tag: 40 | Lexema/Valor: [(]
Linha 12 | Token Tag: 279 | Lexema/Valor: [c]
Linha 12 | Token Tag: 41 | Lexema/Valor: [)]
Linha 12 | Token Tag: 59 | Lexema/Valor: [;]
Linha 13 | Token Tag: 263 | Lexema/Valor: [if]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 279 | Lexema/Valor: [a]
Linha 13 | Token Tag: 62 | Lexema/Valor: [>]
Linha 13 | Token Tag: 279 | Lexema/Valor: [b]
Linha 13 | Token Tag: 41 | Lexema/Valor: [)]
Linha 13 | Token Tag: 292 | Lexema/Valor: [&&]
Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
Linha 13 | Token Tag: 279 | Lexema/Valor: [a]
Linha 13 | Token Tag: 62 | Lexema/Valor: [>]
Linha 13 | Token Tag: 279 | Lexema/Valor: [c]
Linha 13 | Token Tag: 41 | Lexema/Valor: [)]

Linha 13 | Token Tag: 41 | Lexema/Valor: [)]
Linha 13 | Token Tag: 258 | Lexema/Valor: [end]
Linha 14 | Token Tag: 279 | Lexema/Valor: [maior]
Linha 14 | Token Tag: 61 | Lexema/Valor: [=]
Linha 14 | Token Tag: 279 | Lexema/Valor: [a]
Linha 16 | Token Tag: 265 | Lexema/Valor: [else]
Linha 17 | Token Tag: 263 | Lexema/Valor: [if]
Linha 17 | Token Tag: 40 | Lexema/Valor: [(]
Linha 17 | Token Tag: 279 | Lexema/Valor: [b]
Linha 17 | Token Tag: 62 | Lexema/Valor: [>]
Linha 17 | Token Tag: 279 | Lexema/Valor: [c]
Linha 17 | Token Tag: 41 | Lexema/Valor: [)]
Linha 17 | Token Tag: 264 | Lexema/Valor: [then]
Linha 18 | Token Tag: 279 | Lexema/Valor: [maior]
Linha 18 | Token Tag: 61 | Lexema/Valor: [=]
Linha 18 | Token Tag: 279 | Lexema/Valor: [b]
Linha 18 | Token Tag: 59 | Lexema/Valor: [;]
Linha 20 | Token Tag: 265 | Lexema/Valor: [else]
Linha 21 | Token Tag: 279 | Lexema/Valor: [maior]
Linha 21 | Token Tag: 61 | Lexema/Valor: [=]
Linha 21 | Token Tag: 279 | Lexema/Valor: [c]
Linha 22 | Token Tag: 258 | Lexema/Valor: [end]
Linha 23 | Token Tag: 258 | Lexema/Valor: [end]
Linha 23 | Token Tag: 59 | Lexema/Valor: [;]
Linha 24 | Token Tag: 279 | Lexema/Valor: [write]
Linha 24 | Token Tag: 40 | Lexema/Valor: [(]
Linha 24 | Token Tag: 282 | Lexema/Valor: ["Maior valor: "
Linha 24 | Token Tag: 282 | Lexema/Valor: [""]
Linha 24 | Token Tag: 41 | Lexema/Valor: [)]
Linha 24 | Token Tag: 59 | Lexema/Valor: [;]
Linha 25 | Token Tag: 279 | Lexema/Valor: [write]
Linha 25 | Token Tag: 40 | Lexema/Valor: [(]
Linha 25 | Token Tag: 279 | Lexema/Valor: [maior]
Linha 25 | Token Tag: 41 | Lexema/Valor: [)]
Linha 25 | Token Tag: 59 | Lexema/Valor: [;]
Linha 26 | Token Tag: 279 | Lexema/Valor: [write]
Linha 26 | Token Tag: 40 | Lexema/Valor: [(]
Linha 26 | Token Tag: 282 | Lexema/Valor: [" Continuar?(S/N) "
Linha 26 | Token Tag: 41 | Lexema/Valor: [)]
Linha 26 | Token Tag: 59 | Lexema/Valor: [;]
Linha 27 | Token Tag: 279 | Lexema/Valor: [read]
Linha 27 | Token Tag: 40 | Lexema/Valor: [(]
Linha 27 | Token Tag: 279 | Lexema/Valor: [outro]
Linha 27 | Token Tag: 41 | Lexema/Valor: [)]
Linha 27 | Token Tag: 59 | Lexema/Valor: [;]
Linha 28 | Token Tag: 269 | Lexema/Valor: [until]
Linha 28 | Token Tag: 40 | Lexema/Valor: [(]
Linha 28 | Token Tag: 279 | Lexema/Valor: [opc]

Linha 28 | Token Tag: 288 | Lexema/Valor: [==]
 Linha 28 | Token Tag: 281 | Lexema/Valor: ['N']
 Linha 28 | Token Tag: 293 | Lexema/Valor: [||]
 Linha 28 | Token Tag: 279 | Lexema/Valor: [opc]
 Linha 28 | Token Tag: 288 | Lexema/Valor: [==]
 Linha 28 | Token Tag: 281 | Lexema/Valor: ['n']
 Linha 28 | Token Tag: 41 | Lexema/Valor: []]
 Linha 29 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 29 | Token Tag: 46 | Lexema/Valor: [.]

TABELA DE SIMBOLOS

Lexema: int | Tag: 260
 Lexema: teste5 | Tag: 279
 Lexema: maior | Tag: 279
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: program | Tag: 256
 Lexema: in | Tag: 270
 Lexema: opc | Tag: 279
 Lexema: repeat | Tag: 268
 Lexema: outro | Tag: 279
 Lexema: if | Tag: 263
 Lexema: while | Tag: 266
 Lexema: do | Tag: 267
 Lexema: read | Tag: 279
 Lexema: out | Tag: 271
 Lexema: else | Tag: 265
 Lexema: write | Tag: 279
 Lexema: then | Tag: 264
 Lexema: char | Tag: 261
 Lexema: c | Tag: 279
 Lexema: begin | Tag: 257
 Lexema: b | Tag: 279
 Lexema: a | Tag: 279
 Lexema: float | Tag: 262

TESTE 6.1

Programa fonte analisado:

```

program
int: a, b;
float: resultado;
char: opcao;
begin
    /* comentario de uma linha */
    write("Digite o valor de a:");
    read(a);
    opcao = 'S';
    b = 10;
    resultado = 345.27;
    if (a >= b && opcao != 'N') then
        write("Processando");
    end
end.

```

Resultado: SUCESSO!

----- SEQUENCIA DE TOKENS -----

Linha 1 | Token Tag: 256 | Lexema/Valor: [program]
 Linha 2 | Token Tag: 260 | Lexema/Valor: [int]
 Linha 2 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [a]
 Linha 2 | Token Tag: 44 | Lexema/Valor: [,]
 Linha 2 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 2 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 3 | Token Tag: 262 | Lexema/Valor: [float]
 Linha 3 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 3 | Token Tag: 279 | Lexema/Valor: [resultado]
 Linha 3 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 4 | Token Tag: 261 | Lexema/Valor: [char]
 Linha 4 | Token Tag: 58 | Lexema/Valor: [:]
 Linha 4 | Token Tag: 279 | Lexema/Valor: [opcao]
 Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 5 | Token Tag: 257 | Lexema/Valor: [begin]
 Linha 7 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 7 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 7 | Token Tag: 282 | Lexema/Valor: ["Digite o valor de a:"]
 Linha 7 | Token Tag: 41 | Lexema/Valor: [)]
 Linha 7 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 8 | Token Tag: 279 | Lexema/Valor: [read]
 Linha 8 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 8 | Token Tag: 279 | Lexema/Valor: [a]

Linha 8 | Token Tag: 41 | Lexema/Valor: []
 Linha 8 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 9 | Token Tag: 279 | Lexema/Valor: [opcao]
 Linha 9 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 9 | Token Tag: 281 | Lexema/Valor: ['S']
 Linha 9 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 10 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 10 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 10 | Token Tag: 278 | Lexema/Valor: [10]
 Linha 10 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 11 | Token Tag: 279 | Lexema/Valor: [resultado]
 Linha 11 | Token Tag: 61 | Lexema/Valor: [=]
 Linha 11 | Token Tag: 280 | Lexema/Valor: [345.27]
 Linha 11 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 12 | Token Tag: 263 | Lexema/Valor: [if]
 Linha 12 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 12 | Token Tag: 279 | Lexema/Valor: [a]
 Linha 12 | Token Tag: 289 | Lexema/Valor: [>=]
 Linha 12 | Token Tag: 279 | Lexema/Valor: [b]
 Linha 12 | Token Tag: 292 | Lexema/Valor: [&&]
 Linha 12 | Token Tag: 279 | Lexema/Valor: [opcao]
 Linha 12 | Token Tag: 291 | Lexema/Valor: [!=]
 Linha 12 | Token Tag: 281 | Lexema/Valor: ['N']
 Linha 12 | Token Tag: 41 | Lexema/Valor: []
 Linha 12 | Token Tag: 264 | Lexema/Valor: [then]
 Linha 13 | Token Tag: 279 | Lexema/Valor: [write]
 Linha 13 | Token Tag: 40 | Lexema/Valor: [(]
 Linha 13 | Token Tag: 282 | Lexema/Valor: ["Processando"]
 Linha 13 | Token Tag: 41 | Lexema/Valor: []
 Linha 13 | Token Tag: 59 | Lexema/Valor: [;]
 Linha 14 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 15 | Token Tag: 258 | Lexema/Valor: [end]
 Linha 15 | Token Tag: 46 | Lexema/Valor: [.]

TABELA DE SIMBOLOS

Lexema: int | Tag: 260
 Lexema: end | Tag: 258
 Lexema: until | Tag: 269
 Lexema: program | Tag: 256
 Lexema: in | Tag: 270
 Lexema: repeat | Tag: 268
 Lexema: if | Tag: 263
 Lexema: opcao | Tag: 279
 Lexema: while | Tag: 266
 Lexema: do | Tag: 267
 Lexema: read | Tag: 279

Lexema: out | Tag: 271
Lexema: else | Tag: 265
Lexema: write | Tag: 279
Lexema: then | Tag: 264
Lexema: char | Tag: 261
Lexema: begin | Tag: 257
Lexema: b | Tag: 279
Lexema: a | Tag: 279
Lexema: resultado | Tag: 279
Lexema: float | Tag: 262

TESTE 6.2

Programa fonte analisado:

```
program
| float: area$;
begin
| out("Teste com erro lexico");
end
```

Resultado:

Erro na leitura do arquivo: Erro lexico (linha 2): caractere invalido '\$'.

Correção 1:

```
program
float: area_val;
begin
out("Teste com erro lexico");
end
```

Resultado: SUCESSO!

SEQUENCIA DE TOKENS

Linha 1 | Token Tag: 256 | Lexema/Valor: [program]
Linha 2 | Token Tag: 262 | Lexema/Valor: [float]
Linha 2 | Token Tag: 58 | Lexema/Valor: [:]

Linha 2 | Token Tag: 279 | Lexema/Valor: [area_val]
Linha 2 | Token Tag: 59 | Lexema/Valor: [;]
Linha 3 | Token Tag: 257 | Lexema/Valor: [begin]
Linha 4 | Token Tag: 271 | Lexema/Valor: [out]
Linha 4 | Token Tag: 40 | Lexema/Valor: [(]
Linha 4 | Token Tag: 282 | Lexema/Valor: ["Teste com erro lexico"]
Linha 4 | Token Tag: 41 | Lexema/Valor: [)]
Linha 4 | Token Tag: 59 | Lexema/Valor: [;]
Linha 5 | Token Tag: 258 | Lexema/Valor: [end]

TABELA DE SIMBOLOS

Lexema: if | Tag: 263
Lexema: char | Tag: 261
Lexema: until | Tag: 269
Lexema: repeat | Tag: 268
Lexema: begin | Tag: 257
Lexema: float | Tag: 262
Lexema: end | Tag: 258
Lexema: out | Tag: 271
Lexema: while | Tag: 266
Lexema: do | Tag: 267
Lexema: program | Tag: 256
Lexema: then | Tag: 264
Lexema: area_val | Tag: 279
Lexema: int | Tag: 260
Lexema: in | Tag: 270
Lexema: else | Tag: 265
